



Cloud Computing Lab 10

Submitted To:

Sir Shoaib & Sir Waqas

Submitted By:

Hamail Fatima

Roll No:

2023-BSE-023

Section:

5(A)

Task 1 — GitHub CLI Codespace Setup & Authentication

Goal: Install GH CLI (if not present), authenticate for Codespace use, and connect into the Codespace shell (**all steps inside Codespace unless noted**).

1. Install GitHub CLI:

winget install --id GitHub.cli

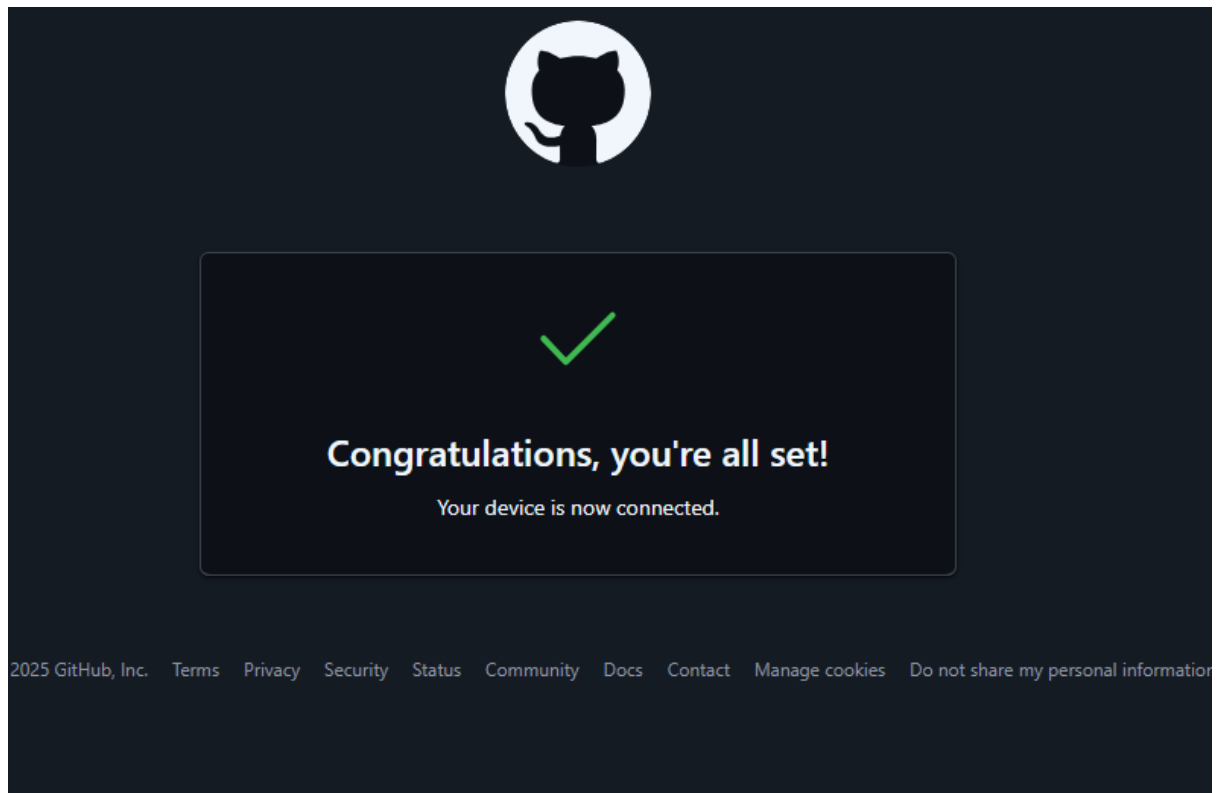
- **Save screenshot as:** task1_gh_install.png — terminal showing the winget install command output.

```
PS C:\Users\admin> winget install --id GitHub.cli
Found an existing package already installed. Trying to upgrade the installed package...
No available upgrade found.
No newer package versions are available from the configured sources.
PS C:\Users\admin> |
```

2. Authenticate GH CLI for Codespaces:

gh auth login -s codespace

- When prompted, generate a **GitHub access token (classic)**. Set scopes to: admin:org, codespace, repo.
- **Save screenshot as:** task1_gh_auth_login.png — screenshot of gh auth login confirmation (redact token).



```

! First copy your one-time code: 5BBC-E6E0
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as Hamail-maker
! You were already logged in to this account
PS C:\Users\admin> |

```

3. List available codespaces:

gh codespace list

- **Save screenshot as:** task1_codespace_list.png — output showing the codespace name.

```

PS C:\Users\admin> gh codespace list
NAME                DISPLAY_NAME  REPOSITORY      BRANCH  STATE      CREATED AT
potential-eureka-q7xw45r4wg5vhpq  potential eureka  Hamail-maker/my-lab-repo  main*   Shutdown   about 20 hours ago
PS C:\Users\admin> |

```

4. Connect to a codespace via SSH:

gh codespace ssh -c <name_of_codespace>

- **Save screenshot as:** task1_codespace_ssh_connected.png — terminal inside the Codespace shell after ssh.

```

PS C:\Users\admin> gh codespace ssh -c potential-eureka-q7xw45r4wg5vhpq
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Last login: Tue Dec 16 13:45:23 2025 from ::1
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

Task 2 — Install AWS CLI, Terraform CLI, Provider Setup

Goal: Install AWS CLI (if not present) & Terraform CLI inside Codespace; configure initial Terraform provider.

A. Install AWS CLI (Skip if already installed)

1. In Codespace shell, install AWS CLI:
2. `curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"`
3. `unzip awscliv2.zip`
4. `sudo ./aws/install`
`aws --version`

- **Save screenshot as:** task2_aws_install_and_version.png — show AWS CLI installation and version output.

```
inflating: aws/dist/wheel-0.45.1.dist-info/METADATA
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws --version
aws-cli/2.32.17 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

5. Configure AWS CLI:

aws configure

- **Save screenshot as:** task2_aws_configure_and_files.png — show aws configure prompt (redact secret values).

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws configure
AWS Access Key ID [*****35UE]: AKIA42E3QACSJXZJ35UE
AWS Secret Access Key [*****A3V9]: 0V+BGMfMvkcKRSHz6ztGZRioMVqJXKeDfL/qA3V9
Default region name [me-central-1]: me-central-1
Default output format [json]: json
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

- Then check config files:

cat ~/.aws/credentials

cat ~/.aws/config

- **Save screenshot as:** task2_aws_configure_and_files.png — output of cat commands (redact secret keys if visible).

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ cat ~/.aws/credentials
cat ~/.aws/config
[default]
aws_access_key_id = AKIA42E3QACSJXZJ35UE
aws_secret_access_key = 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
[default]
region = me-central-1
output = json
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

6. Verify AWS CLI connectivity:

aws sts get-caller-identity

- **Save screenshot as:** task2_aws_get_caller_identity.png — output of aws sts get-caller-identity.

```
@Hamaill-maker → /workspaces/my-lab-repo (main) $ aws sts get-caller-identity
{
  "UserId": "AIDA42E3QACSNIPKW2KQG",
  "Account": "880794402980",
  "Arn": "arn:aws:iam::880794402980:user/lab-user"
}
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

B. Install Terraform CLI

1. Install Terraform CLI:
 2. `wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg`
 3. `echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list`
 4. `sudo apt update`
 5. `sudo apt install terraform`
 6. `which terraform`
`terraform --version`
- **Save screenshot as:** task2_terraform_install_and_version.png — terraform installation and version check output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ sudo apt install terraform
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  terraform
0 upgraded, 1 newly installed, 0 to remove and 49 not upgraded.
Need to get 30.6 MB of archives.
After this operation, 101 MB of additional disk space will be used.
Get:1 https://apt.releases.hashicorp.com noble/main amd64 terraform amd64 1.14.2-1 [30.6 MB]
Fetched 30.6 MB in 0s (146 MB/s)
Selecting previously unselected package terraform.
(Reading database ... 58629 files and directories currently installed.)
Preparing to unpack ../terraform_1.14.2-1_amd64.deb ...
Unpacking terraform (1.14.2-1) ...
Setting up terraform (1.14.2-1) ...
@Hamail-maker → /workspaces/my-lab-repo (main) $ which terraform
terraform --version
/usr/bin/terraform
Terraform v1.14.2
on linux_amd64
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

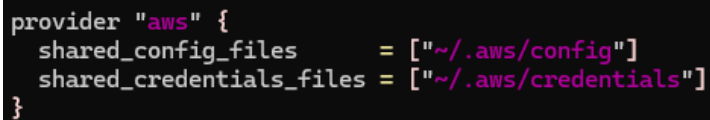
C. Provider Configuration (main.tf)

- Create main.tf using Vim:
`vim main.tf`
- **Save screenshot as:** task2_provider_file_creation.png — creation/editing of main.tf.



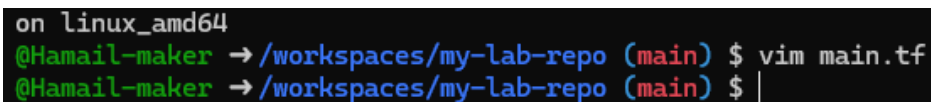
```
"main.tf" [New]
```

- Inside main.tf, write:
 - provider "aws" {
 - shared_config_files = ["~/.aws/config"]
 - shared_credentials_files = ["~/.aws/credentials"]
 - }
- **Save screenshot as:** task2_provider_block.png — final content saved in main.tf.



```
provider "aws" {  
  shared_config_files = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}
```

- Save and quit: Press ESC then :wq
- **Save screenshot as:** task2_vim_save_main_tf.png — Vim save confirmation (optional).



```
on linux_amd64  
@Hamaill-maker → /workspaces/my-lab-repo (main) $ vim main.tf  
@Hamaill-maker → /workspaces/my-lab-repo (main) $ |
```

- Initialize Terraform:
- ```
terraform init
```
- **Save screenshot as:** task2\_terraform\_init\_output.png — terraform init output.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.26.0...
- Installed hashicorp/aws v6.26.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

- Show contents of .terraform.lock.hcl:

```
cat .terraform.lock.hcl
```

- Save screenshot as: task2\_terraform\_lock\_hcl.png — output of .terraform.lock.hcl.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ cat .terraform.lock.hcl
This file is maintained automatically by "terraform init".
Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
 version = "6.26.0"
 hashes = [
 "h1:B7X8EU6aZ9KzIvP0VBDhghdgjXIrUgMXt/pJ6EUXvo=",
 "zh:038fd943de79acd9f9f73106fa0eba588c6a0d4e0993e146f51f3aa043728c5f",
 "zh:06fa0177d33d3d3f9cb7e205fbeb1c4c3095ba637e2b4d292429401ec5612e81",
 "zh:212714fc8b6ee57e26d11d0fdf2ecfe23b37a6eac1008b399c1d790528c3f072",
 "zh:3197725d772f360e9e466b68a5ba67363e9f6786809c9adefc50f7f7e525bf42",
 "zh:33385539f3e3fafb96c6036421fd72b05c76505eeefaaff8a089c3eeba25db65",
 "zh:4ce065e0d3c384d11c1b59fe92582d331aae27ff6e019ace07b8cedef5653aae",
 "zh:67863d6ff5517db2c0b8097443708dca548f1922d2e08ad76a98d493ff480cb1",
 "zh:771ccf61fc107013b437b0a05cdb342823a99200653bfe9b892702b9fd8997fe",
 "zh:80adcf83bef9d683606c48bbe53cbb2b5a04878641674e957939b5e8f124ada0",
 "zh:9675c7f209db8e64ba2d5197acc8ba0073bd73b48c3dd61a1961a44844bc8a81",
 "zh:9b12af85486a96aedd8d7984b0ff811a4b42e3d88dad1a3fb4c0b580d04fa425",
 "zh:b47d0f5eff91c94c5d5677815b9361e64dfbe2ee2d59ba2867e2d0f5fa7181e4",
 "zh:b4933663b8d6cc1cfb51aa47bd8f26c06012ee2e278e57663affdc722dd5baa",
 "zh:d53a94ecdb6b68a8dc19ec6e16ba2d4c2acde575af254d1b8b80143e57c76abf",
 "zh:e7cb8c1770c6f87c5ce1d3e28b838380bb8e5296dd03034b796168de8be1c7ec",
]
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

- Show contents of .terraform/ directory:

```
ls .terraform/
```

- Save screenshot as: task2\_terraform\_dir\_ls.png — output of ls .terraform/.

```
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ ls .terraform/
providers
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

### Task 3 — VPC/Subnet Creation, Initialization, Verification

Goal: Use Terraform to create VPC & Subnet resources, verify via CLI.

1. Edit main.tf to add:
  2. resource "aws\_vpc" "development\_vpc" {
  3.   cidr\_block = "10.0.0.0/16"
  4. }
  - 5.
  6. resource "aws\_subnet" "dev\_subnet\_1" {
  7.   vpc\_id    = aws\_vpc.development\_vpc.id
  8.   cidr\_block = "10.0.10.0/24"
  9.   availability\_zone = "me-central-1a"
  - }
- Use vim main.tf, edit and insert code.
  - **Save screenshot as:** task3\_main\_tf\_resource\_add.png — new resources added and visible in main.tf.

```
provider "aws" {
 shared_config_files = ["~/.aws/config"]
 shared_credentials_files = ["~/.aws/credentials"]
}
|
~
~
~
~
~
```

```
provider "aws" {
 shared_config_files = ["~/.aws/config"]
 shared_credentials_files = ["~/.aws/credentials"]
}
resource "aws_vpc" "development_vpc" {
 cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev_subnet_1" {
 vpc_id = aws_vpc.development_vpc.id
 cidr_block = "10.0.10.0/24"
 availability_zone = "me-central-1a"
}
|
```



10. Run:

terraform apply

- Type yes when prompted.
- **Save screenshot as:** task3\_terraform\_apply\_vpc\_subnet.png — output showing successful apply and resources creation.

```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.development_vpc: Creating...
aws_vpc.development_vpc: Creation complete after 2s [id=vpc-0d83f962cfc1ff84c]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 1s [id=subnet-03826069b6b18045c]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

11. Verify resources using AWS CLI:

- Run:  
aws ec2 describe-subnets --filter "Name=subnet-id,Values=<subnet-id>"
- **Save screenshot as:** task3\_aws\_cli\_verify\_subnet.png — output of describe-subnets command.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-subnets --filter "Name=subnet-id,Values=subnet-03826069b6b18045c"
{
 "Subnets": [
 {
 "AvailabilityZoneId": "mec1-az1",
 "MapCustomerOwnedIpOnLaunch": false,
 "OwnerId": "880794402980",
 "AssignIpv6AddressOnCreation": false,
 "Ipv6CidrBlockAssociationSet": [],
 "SubnetArn": "arn:aws:ec2:me-central-1:880794402980:subnet/subnet-03826069b6b18045c",
 "EnableDns64": false,
 "Ipv6Native": false,
 "PrivateDnsNameOptionsOnLaunch": {
 "HostnameType": "ip-name",
 "EnableResourceNameDnsARecord": false,
 "EnableResourceNameDnsAAAARecord": false
 },
 "BlockPublicAccessStates": {
 "InternetGatewayBlockMode": "off"
 },
 "SubnetId": "subnet-03826069b6b18045c",
 "State": "available",
 "VpcId": "vpc-0d83f962cfc1ff84c",
 "CidrBlock": "10.0.10.0/24",
 "AvailableIpAddressCount": 251,
 "AvailabilityZone": "me-central-1a",
 "DefaultForAz": false,
 }
]
}
```

- Run:  
aws ec2 describe-vpcs --filter "Name=vpc-id,Values=<vpc-id>"
- **Save screenshot as:** task3\_aws\_cli\_verify\_vpc.png — output of describe-vpcs command.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ aws ec2 describe-vpcs --filter "Name=vpc-id,Values=vpc-0d83f962cfc1ff84c"
{
 "Vpcs": [
 {
 "OwnerId": "880794402980",
 "InstanceTenancy": "default",
 "CidrBlockAssociationSet": [
 {
 "AssociationId": "vpc-cidr-assoc-09a1c0c2ec0c726fb",
 "CidrBlock": "10.0.0.0/16",
 "CidrBlockState": {
 "State": "associated"
 }
 }
],
 "IsDefault": false,
 "BlockPublicAccessStates": {
 "InternetGatewayBlockMode": "off"
 },
 "VpcId": "vpc-0d83f962cfc1ff84c",
 "State": "available",
 "CidrBlock": "10.0.0.0/16",
 "DhcpOptionsId": "dopt-00cbdc96c2de8964b"
 }
]
}
```

## Task 4 — Data Source, Targeted Destroy, Tags

### A. Data Source & Resource Creation

1. Add to main.tf:
2. data "aws\_vpc" "existing\_vpc" {
3.   default = true
4. }
- 5.
6. resource "aws\_subnet" "dev\_subnet\_1\_existing" {
7.   vpc\_id   = data.aws\_vpc.existing\_vpc.id
8.   cidr\_block = "172.31.48.0/24"
9.   availability\_zone = "me-central-1a"
- }
- Use vim to edit and insert code.
- **Save screenshot as:** task4\_main\_tf\_datasource\_resource\_add.png — main.tf with datasource and resource added.

```

provider "aws" {
 shared_config_files = ["~/.aws/config"]
 shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "development_vpc" {
 cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev_subnet_1" {
 vpc_id = aws_vpc.development_vpc.id
 cidr_block = "10.0.10.0/24"
 availability_zone = "me-central-1a"
}

data "aws_vpc" "existing_vpc" {
 default = true
}

resource "aws_subnet" "dev_subnet_1_existing" {
 vpc_id = data.aws_vpc.existing_vpc.id
 cidr_block = "172.31.48.0/24"
 availability_zone = "me-central-1a"
}

```

#### 10. Apply configuration:

terraform apply

- Confirm only the new subnet is created.
- **Save screenshot as:** task4\_terraform\_apply\_datasource\_resource.png — terraform output of resource creation.

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
 Terraform will perform the actions described above.
 Only 'yes' will be accepted to approve.

 Enter a value: yes

aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-02821ba796b347c6d]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

#### B. Targeted Destroy & Refresh

##### 1. Destroy only one resource:

terraform destroy -target=aws\_subnet.dev\_subnet\_1\_existing

- **Save screenshot as:** task4\_terraform\_destroy\_targeted.png — targeted destroy output.

```
Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been
ignored and the output values may not be fully updated. Run the following command to verify that no other changes are
pending:
 terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as
recovering from errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Destroy complete! Resources: 1 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

## 2. Refresh state:

terraform refresh

- **Save screenshot as:** task4\_terraform\_refresh\_state.png — terraform refresh output.

```
Destroy complete! Resources: 1 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ terraform refresh
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development_vpc: Refreshing state... [id=vpc-0d83f962cfc1ff84c]
data.aws_vpc.existing_vpc: Read complete after 1s [id=vpc-02e5f1f03e6b88a38]
aws_subnet.dev_subnet_1: Refreshing state... [id=subnet-03826069b6b18045c]
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

## 3. Re-apply configuration:

terraform apply

- **Save screenshot as:** task4\_terraform\_apply\_after\_refresh.png — terraform apply after refresh.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
 Terraform will perform the actions described above.
 Only 'yes' will be accepted to approve.

 Enter a value: yes

aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-066eb99b95123e0ec]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

## 4. Destroy all resources:

terraform destroy

- **Save screenshot as:** task4\_terraform\_destroy\_all.png — full destroy output.

```

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
 Terraform will destroy all your managed infrastructure, as shown above.
 There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1: Destroying... [id=subnet-03826069b6b18045c]
aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-066eb99b95123e0ec]
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-0d83f962cfc1ff84c]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_vpc.development_vpc: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

##### 5. Plan configuration:

terraform plan

- **Save screenshot as:** task4\_terraform\_plan\_output.png — terraform plan showing next changes.

```

aws_vpc.development_vpc will be created
+ resource "aws_vpc" "development_vpc" {
 + arn = (known after apply)
 + cidr_block = "10.0.0.0/16"
 + default_network_acl_id = (known after apply)
 + default_route_table_id = (known after apply)
 + default_security_group_id = (known after apply)
 + dhcp_options_id = (known after apply)
 + enable_dns_hostnames = (known after apply)
 + enable_dns_support = true
 + enable_network_address_usage_metrics = (known after apply)
 + id = (known after apply)
 + instance_tenancy = "default"
 + ipv6_association_id = (known after apply)
 + ipv6_cidr_block = (known after apply)
 + ipv6_cidr_block_network_border_group = (known after apply)
 + main_route_table_id = (known after apply)
 + owner_id = (known after apply)
 + region = "me-central-1"
 + tags_all = (known after apply)
}

Plan: 3 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee you run "terraform apply" now.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

##### 6. Apply again:

terraform apply

- **Save screenshot as:** task4\_terraform\_apply\_after\_destroy.png — output showing resources recreated.

```
Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
 Terraform will perform the actions described above.
 Only 'yes' will be accepted to approve.

 Enter a value: yes

aws_subnet.dev_subnet_1_existing: Creating...
aws_vpc.development_vpc: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-0e1d7c2d5153a327c]
aws_vpc.development_vpc: Creation complete after 2s [id=vpc-0795b0c67757c14a8]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 0s [id=subnet-0b16ea6dbb5b176c4]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

### C. Tagging Resources

1. Modify main.tf to add tags:
2. resource "aws\_vpc" "development\_vpc" {
3. cidr\_block = "10.0.0.0/16"
4. tags = {
5. Name: "development"
6. vpc\_env = "dev"
7. }
8. }
- 9.
10. resource "aws\_subnet" "dev\_subnet\_1" {
11. vpc\_id = aws\_vpc.development\_vpc.id
12. cidr\_block = "10.0.10.0/24"
13. availability\_zone = "me-central-1a"
14. tags = {
15. Name: "subnet-1-dev"
16. }
17. }

18.

19. resource "aws\_subnet" "dev\_subnet\_1\_existing" {

20. vpc\_id = data.aws\_vpc.existing\_vpc.id

21. cidr\_block = "172.31.48.0/24"

22. availability\_zone = "me-central-1a"

23. tags = {

24. Name: "subnet-1-default"

25. }

}

- Use vim to update tags.
- **Save screenshot as:** task4\_main\_tf\_tagging.png — main.tf with tags added.

```
tags = {
 Name = "development"
 vpc_env = "dev"
}

resource "aws_subnet" "dev_subnet_1" {
 vpc_id = aws_vpc.development_vpc.id
 cidr_block = "10.0.10.0/24"
 availability_zone = "me-central-1a"

 tags = {
 Name = "subnet-1-dev"
 }
}

resource "aws_subnet" "dev_subnet_1_existing" {
 vpc_id = data.aws_vpc.existing_vpc.id
 cidr_block = "172.31.48.0/24"
 availability_zone = "me-central-1a"

 tags = {
 Name = "subnet-1-default"
 }
}
```

26. Run:

27. terraform refresh

terraform apply -auto-approve

- **Save screenshot as:** task4\_terraform\_apply\_tagging.png — output showing tags applied.

```

}

aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
 id = "vpc-0795b0c67757c14a8"
 ~ tags = {
 + "Name" = "development"
 + "vpc_env" = "dev"
 }
 ~ tags_all = {
 + "Name" = "development"
 + "vpc_env" = "dev"
 }
 # (19 unchanged attributes hidden)
}

Plan: 0 to add, 3 to change, 0 to destroy.
aws_subnet.dev_subnet_1_existing: Modifying... [id=subnet-0e1d7c2d5153a327c]
aws_vpc.development_vpc: Modifying... [id=vpc-0795b0c67757c14a8]
aws_subnet.dev_subnet_1_existing: Modifications complete after 0s [id=subnet-0e1d7c2d5153a327c]
aws_vpc.development_vpc: Modifications complete after 0s [id=vpc-0795b0c67757c14a8]
aws_subnet.dev_subnet_1: Modifying... [id=subnet-0b16ea6dbb5b176c4]
aws_subnet.dev_subnet_1: Modifications complete after 1s [id=subnet-0b16ea6dbb5b176c4]

Apply complete! Resources: 0 added, 3 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

28. Remove vpc\_env = "dev" tag from development\_vpc resource, re-plan/apply:

- **Save screenshot as:** task4\_terraform\_plan\_remove\_tag.png — plan output showing tag removal.

```

Terraform will perform the following actions:

aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
 id = "vpc-0795b0c67757c14a8"
 ~ tags = {
 "Name" = "development"
 - "vpc_env" = "dev" -> null
 }
 ~ tags_all = {
 - "vpc_env" = "dev" -> null
 # (1 unchanged element hidden)
 }
 # (19 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take ex
you run "terraform apply" now.
@Hamail-maker → /workspaces/my-lab-repo (main) $
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

- **Save screenshot as:** task4\_terraform\_apply\_remove\_tag.png — apply output showing tag deleted.



```

Terraform will perform the following actions:

aws_vpc.development_vpc will be updated in-place
~ resource "aws_vpc" "development_vpc" {
 id = "vpc-0795b0c67757c14a8"
 ~ tags = {
 "Name" = "development"
 - "vpc_env" = "dev" -> null
 }
 ~ tags_all = {
 - "vpc_env" = "dev" -> null
 # (1 unchanged element hidden)
 }
 # (19 unchanged attributes hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
 Terraform will perform the actions described above.
 Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.development_vpc: Modifying... [id=vpc-0795b0c67757c14a8]
aws_vpc.development_vpc: Modifications complete after 1s [id=vpc-0795b0c67757c14a8]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $ |

```

## Task 5 — State File Inspection & Terraform State Commands

1. Destroy all resources:

terraform destroy

- Save screenshot as: task5\_terraform\_destroy.png — destroy output.

```

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
 Terraform will destroy all your managed infrastructure, as shown above.
 There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-0e1d7c2d5153a327c]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-0b16ea6dbb5b176c4]
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-0795b0c67757c14a8]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_vpc.development_vpc: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.

```

2. Inspect state files after destroy:

- Run:

cat terraform.tfstate

- Save screenshot as: task5\_terraform\_state\_file\_empty.png — output showing terraform.tfstate is empty after destroy.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ cat terraform.tfstate
{
 "version": 4,
 "terraform_version": "1.14.2",
 "serial": 30,
 "lineage": "c3816f98-82be-2ed3-d1ec-40cb9a0c9640",
 "outputs": {},
 "resources": [],
 "check_results": null
}
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

- Run:

cat terraform.tfstate.backup

- **Save screenshot as:** task5\_terraform\_state\_backup\_prev.png — output showing backup file with previous resources.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ cat terraform.tfstate.backup
{
 "version": 4,
 "terraform_version": "1.14.2",
 "serial": 25,
 "lineage": "c3816f98-82be-2ed3-d1ec-40cb9a0c9640",
 "outputs": {},
 "resources": [
 {
 "mode": "data",
 "type": "aws_vpc",
 "name": "existing_vpc",
 "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
 "instances": [
 {
 "schema_version": 0,
 "attributes": {
 "arn": "arn:aws:ec2:me-central-1:880794402980:vpc/vpc-02e5f1f03e6b88a38",
```

3. Recreate resources:

terraform apply

- **Save screenshot as:** task5\_terraform\_apply\_recreated.png — apply output, resources recreated.

```
Enter a value: yes
aws_vpc.development_vpc: Creating...
aws_subnet.dev_subnet_1_existing: Creating...
aws_subnet.dev_subnet_1_existing: Creation complete after 1s [id=subnet-01e3ceb4a9a2a5748]
aws_vpc.development_vpc: Creation complete after 2s [id=vpc-0d2d1cb9bd254973c]
aws_subnet.dev_subnet_1: Creating...
aws_subnet.dev_subnet_1: Creation complete after 0s [id=subnet-0c823b19b9f4e7495]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/my-lab-repo (main) $
```

4. View state files:

- Run:

cat terraform.tfstate

- **Save screenshot as:** task5\_terraform\_state\_file\_populated.png — output showing populated terraform.tfstate after restore.

```
@Hmail-maker → /workspaces/my-lab-repo (main) $ cat terraform.tfstate
{
 "version": 4,
 "terraform_version": "1.14.2",
 "serial": 34,
 "lineage": "c3816f98-82be-2ed3-d1ec-40cb9a0c9640",
 "outputs": {},
 "resources": [
 {
 "mode": "data",
 "type": "aws_vpc",
 "name": "existing_vpc",
 "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
 "instances": [
 {
 "schema_version": 0,
 "attributes": {
 "arn": "arn:aws:ec2:me-central-1:880794402980:vpc/vpc-02e5f1f03e6b88a38",
 "cidr_block": "172.31.0.0/16",
 "cidr_block_associations": [
 {
 "association_id": "vpc-cidr-assoc-0f7f2af88f5d889fa",
 "cidr_block": "172.31.0.0/16",
 "state": "associated"
 }
]
 },
 "default": true,

```

- Run:

cat terraform.tfstate.backup

- **Save screenshot as:** task5\_terraform\_state\_backup\_empty.png — output showing backup file empty after restore.

```
@Hmail-maker → /workspaces/my-lab-repo (main) $ cat terraform.tfstate.backup
{
 "version": 4,
 "terraform_version": "1.14.2",
 "serial": 30,
 "lineage": "c3816f98-82be-2ed3-d1ec-40cb9a0c9640",
 "outputs": {},
 "resources": [],
 "check_results": null
}
@Hmail-maker → /workspaces/my-lab-repo (main) $
```

5. List resources:

terraform state list

- **Save screenshot as:** task5\_terraform\_state\_list.png — terraform state list output.

```
@Hmail-maker → /workspaces/my-lab-repo (main) $ terraform state list
data.aws_vpc.existing_vpc
aws_subnet.dev_subnet_1
aws_subnet.dev_subnet_1_existing
aws_vpc.development_vpc
@Hmail-maker → /workspaces/my-lab-repo (main) $
```

6. Show full attributes:

terraform state show <resource-name>

- **Save screenshot as:** task5\_terraform\_state\_show\_resource.png — state show output for one resource.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ terraform state show aws_vpc.development_vpc
aws_vpc.development_vpc:
resource "aws_vpc" "development_vpc" {
 arn = "arn:aws:ec2:me-central-1:880794402980:vpc/vpc-0d2d1cb9bd254973c"
 assign_generated_ipv6_cidr_block = false
 cidr_block = "10.0.0.0/16"
 default_network_acl_id = "acl-0362c1afbc343270c"
 default_route_table_id = "rtb-0b140487561311ccd"
 default_security_group_id = "sg-09b0ee407a9b40c33"
 dhcp_options_id = "dopt-00cbdc96c2de8964b"
 enable_dns_hostnames = false
 enable_dns_support = true
 enable_network_address_usage_metrics = false
 id = "vpc-0d2d1cb9bd254973c"
 instance_tenancy = "default"
 ipv6_association_id = null
 ipv6_cidr_block = null
 ipv6_cidr_block_network_border_group = null
 ipv6_ipam_pool_id = null
}
```

7. Note: Do **NOT** run the following (for information only):

8. terraform state rm <resource-name>

- Mentioned for knowledge — do not run.

```
@Hamail-maker → /workspaces/my-lab-repo (main) $ terraform state rm <resource-name>
```

## Task 6 — Terraform Outputs & Attributes Reporting

1. Add outputs in main.tf:
2. output "dev-vpc-id" {
3. value = aws\_vpc.development\_vpc.id
4. }
5. output "dev-subnet-id" {
6. value = aws\_subnet.dev\_subnet\_1.id
7. }
8. output "dev-vpc-arn" {
9. value = aws\_vpc.development\_vpc.arn
10. }
11. output "dev-subnet-arn" {
12. value = aws\_subnet.dev\_subnet\_1.arn
- }

- **Save screenshot as:** task6\_terraform\_outputs\_basic.png — output values for ids and arns after apply.

```
You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
 Terraform will perform the actions described above.
 Only 'yes' will be accepted to approve.

 Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

dev-subnet-arn = "arn:aws:ec2:me-central-1:880794402980:subnet/subnet-0c823b19b9f4e7495"
dev-subnet-id = "subnet-0c823b19b9f4e7495"
dev-vpc-arn = "arn:aws:ec2:me-central-1:880794402980:vpc/vpc-0d2d1cb9bd254973c"
dev-vpc-id = "vpc-0d2d1cb9bd254973c"
@Hamail-maker → /workspaces/my-lab-repo (main) $ |
```

**Return ALL of the following output attributes from your VPC and Subnet resources:**

- a) cidr\_block
- b) region
- c) tags.Name
- d) tags\_all

Expand output section in main.tf accordingly:

```
output "dev-vpc-cidr_block" {
 value = aws_vpc.development_vpc.cidr_block
}

output "dev-vpc-region" {
 value = aws_vpc.development_vpc.region
}

output "dev-vpc-tags_name" {
 value = aws_vpc.development_vpc.tags["Name"]
}

output "dev-vpc-tags_all" {
 value = aws_vpc.development_vpc.tags_all
}

output "dev-subnet-cidr_block" {
```

```

 value = aws_subnet.dev_subnet_1.cidr_block
}

output "dev-subnet-region" {

 value = aws_subnet.dev_subnet_1.availability_zone
}

output "dev-subnet-tags_name" {

 value = aws_subnet.dev_subnet_1.tags["Name"]
}

output "dev-subnet-tags_all" {

 value = aws_subnet.dev_subnet_1.tags_all
}

```

Run terraform apply and record outputs.

- **Save screenshot as:** task6\_expanded\_outputs.png — output values for each required attribute after apply.

```

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

dev-subnet-arn = "arn:aws:ec2:me-central-1:880794402980:subnet/subnet-0c823b19b9f4e7495"
dev-subnet-cidr_block = "10.0.10.0/24"
dev-subnet-id = "subnet-0c823b19b9f4e7495"
dev-subnet-region = "me-central-1a"
dev-subnet-tags_all = tomap({
 "Name" = "subnet-1-dev"
})
dev-subnet-tags_name = "subnet-1-dev"
dev-vpc-arn = "arn:aws:ec2:me-central-1:880794402980:vpc/vpc-0d2d1cb9bd254973c"
dev-vpc-cidr_block = "10.0.0.0/16"
dev-vpc-id = "vpc-0d2d1cb9bd254973c"
dev-vpc-region = "me-central-1"
dev-vpc-tags_all = tomap({
 "Name" = "development"
})
dev-vpc-tags_name = "development"
@Hmail-maker → /workspaces/my-lab-repo (main) $ |

```

## Cleanup — Delete Resources & State Verification

1. Destroy all resources:

terraform destroy

- **Save screenshot as:** cleanup\_destroy\_resources.png — destroy output.

```

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

aws_subnet.dev_subnet_1_existing: Destroying... [id=subnet-01e3ceb4a9a2a5748]
aws_subnet.dev_subnet_1: Destroying... [id=subnet-0c823b19b9f4e7495]
aws_subnet.dev_subnet_1: Destruction complete after 1s
aws_vpc.development_vpc: Destroying... [id=vpc-0d2d1cb9bd254973c]
aws_subnet.dev_subnet_1_existing: Destruction complete after 1s
aws_vpc.development_vpc: Destruction complete after 1s

Destroy complete! Resources: 3 destroyed.

```

2. Inspect state files:

3. cat terraform.tfstate

cat terraform.tfstate.backup

- **Save screenshot as:** cleanup\_state\_files.png — outputs showing file contents after clean-up.

```

@Hamail-maker → /workspaces/my-lab-repo (main) $ cat terraform.tfstate
cat terraform.tfstate.backup
{
 "version": 4,
 "terraform_version": "1.14.2",
 "serial": 41,
 "lineage": "c3816f98-82be-2ed3-d1ec-40cb9a0c9640",
 "outputs": {},
 "resources": [],
 "check_results": null
}
{
 "version": 4,
 "terraform_version": "1.14.2",
 "serial": 36,
 "lineage": "c3816f98-82be-2ed3-d1ec-40cb9a0c9640",
 "outputs": {
 "dev-subnet-arn": {
 "value": "arn:aws:ec2:me-central-1:880794402980:subnet/subnet-0c823b19b9f4e7495",
 "type": "string"
 },
 "dev-subnet-cidr_block": {
 "value": "10.0.10.0/24",
 "type": "string"
 }
 }
}

```

4. Reapply then compare state and backup files to see differences.

```

@Hamail-maker → /workspaces/my-lab-repo (main) $ terraform apply
cat terraform.tfstate
cat terraform.tfstate.backup
data.aws_vpc.existing_vpc: Reading...
data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-02e5f1f03e6b88a38]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
following symbols:
+ create

Terraform will perform the following actions:

aws_subnet.dev_subnet_1 will be created
+ resource "aws_subnet" "dev_subnet_1" {
 + arn = (known after apply)
 + assign_ipv6_address_on_creation = false
 + availability_zone = "me-central-1a"
 + availability_zone_id = (known after apply)

```